

1. Publisher/Subscriber 노드 한 쌍 작성

publisher는 계속해서 **topic**으로 신호를 전송한다. 그리고 **subscriber**는 맞는 신호를 찾아서 정보를 전송받는다. 이때 DDS가 사용되는데 DDS의 작동원리를 예로 들면 **publisher**가 **/topic**이라는 토픽에 **std_msgs::String** 타입으로 발행한다고 네트워크에게 알리고, **subscriber**의 노드가 켜지면 **subscriber**가 **/topic**을 **std_msgs::String** 타입으로 구독하고 싶다고 알린다. 이렇게 타입과 이름이 일치하면 DDS가 자동으로 둘을 연결시켜 준다. 이를 표현한 **publisher** 가 바로

```
pub_ = create_publisher<std_msgs::msg::String>("topic", 10);
```

이다. 여기서 10은 Qos 큐 depth로 subscriber가 순간적으로 정보를 놓쳤을 때 최대 10개 버퍼까지 쌓아두겠다는 뜻이다.

타이머를 통해 `callback()`할 시간을 정할 수 있다. `subscriber`에서는

```
sub_ = create_subscription<std_msgs::msg::String>( "topic", 10, [this](const std_msgs::msg::String & msg) { callback(msg); });
```

이렇게 코드를 작성하는데 `/topic`을 구독하겠다고 등록하면서 메세지가 도착할 때마다 자동으로 실행될 콜백함수도 같이 등록하게 된다.

msg, string 관련 함수를 사용할 때는 헤더파일을 사용하면 되는데 따로 파일을 만들 필요 없이 ROS2에 설치되어 있는걸 사용할 수 있다. 이 헤더파일을 사용하기 위해서는 CMakeLists.txt에 `find_package(std_msgs REQUIRED)`이런식으로 추가해주면 경로를 읽어오고 `ament_target_dependencies(my_pub rclcpp std_msgs)`를 통해 헤더파일을 사용할 수 있다.

```
[INFO] [1789443433.765899145] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443433.766006699] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443433.766006284] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443433.766020580] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443434.765999899] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443435.266147886] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443435.766100418] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443436.266093741] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443436.766081824] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443437.266027840] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443437.765983395] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443438.266047592] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443438.766057640] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443439.265931335] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443439.766024261] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443440.266042986] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443440.766188537] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443441.266164984] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443441.766019485] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443442.266197473] [my_sub]: 받은 메시지 : 'Hello World!'
^C[INFO] [1789443443.154855393] [rclcpp]: signal_handler(SIGINT/SIGTERM)
yepunna@yepunna:~$ ros2 run my_cpp_pkg my_pub
[INFO] [1789443453.765723744] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443584.192693343] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443584.692898695] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443585.192934350] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443585.692905432] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443586.192761288] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443586.692844651] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443587.192915894] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443587.692902043] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443588.192918575] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443588.692897766] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443589.192906275] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443589.692762844] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443590.192911797] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443590.692902224] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443591.192903451] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443591.692901911] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443592.192907882] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443592.692802977] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443593.192708580] [my_sub]: 받은 메시지 : 'Hello World!'
[INFO] [1789443593.692910801] [my_sub]: 받은 메시지 : 'Hello World!'
yepunna@yepunna:~$ source ~/.ros2_ws/install/setup.bash
yepunna@yepunna:~$ ros2 run my_cpp_pkg my_sub
^C[INFO] [1789443442.383162717] [rclcpp]: signal_handler(SIGINT/SIGTERM)
yepunna@yepunna:~$ cd ~/ros2_ws
colcon build --packages-select my_cpp_pkg
Starting >>> my_cpp_pkg
--- stderr: my_cpp_pkg
In file included from /opt/ros/jazzy/include/rclcpp/rclcpp/logging.hpp:24,
                 from /opt/ros/jazzy/include/rclcpp/rclcpp/copy_all_parameter_values.hpp:27
:
    from /opt/ros/jazzy/include/rclcpp/rclcpp/rclcpp.hpp:175,
    from /home/yepunna/ros2_ws/src/my_cpp_pkg/src/my_cpp_pkg/src/my_sub.cpp:11:
/home/yepunna/ros2_ws/src/my_cpp_pkg/src/my_cpp_pkg: In member function 'void MySub::calltb
ack(const std_msgs::msg::String&)':
/home/yepunna/ros2_ws/src/my_cpp_pkg/src/my_sub.cpp:17:53: error: 's' was not declared in
this scope
   17 |         RCLCPP_INFO(this->get_logger(), "받은 메시지 : \"%s\"", msg.data.c_str());
      |                                     ^
gnake[[2]]: *** [CMakeFiles/my_sub.dir/build.make:76: CMakeFiles/my_sub.dir/src/my_sub.cpp.o] Error 1
gnake[[1]]: *** [CMakeFiles/Makefile2:165: CMakeFiles/my_sub.dir/all] Error 2
gnake: *** [Makefile:146: all] Error 2

Failed <<< my_cpp_pkg [2.78s, exited with code 2]

Summary: 0 packages finished [2.88s]
1 package failed: my_cpp_pkg
1 package had stderr output: my_cpp_pkg
yepunna@yepunna:~/ros2_ws$ colcon build -packages-select my_cpp_pkg
Starting >>> my_cpp_pkg
Finished <<< my_cpp_pkg [5.05s]

Summary: 1 package finished [5.14s]
yepunna@yepunna:~/ros2_ws$ ros2 run my_cpp_pkg my_pub
^C[INFO] [1789443594.913892864] [rclcpp]: signal_handler(SIGINT/SIGTERM)
yepunna@yepunna:~/ros2_ws$
```

publisherd에서 Hello world!를 보내고 subscriber에서 와 헤더파일을 사용할 수 있다.와 헤더파일을 사용할 수 있다.수신하여 “받은 메세지 : “Hello world!””를 출력한다.

2. launch 파일로 두 노드 동시 실행

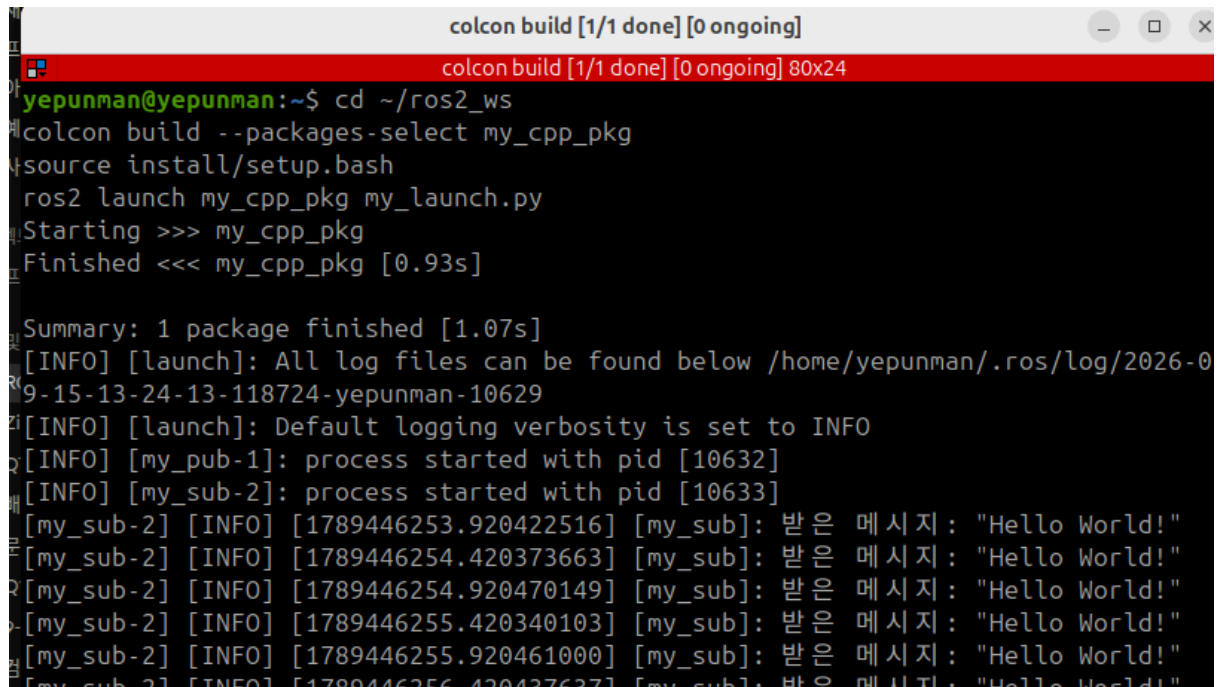
launch는 여러개의 노드를 한번에 실행 시켜 준다. 코드는 다음과 같다.

```
def generate_launch_description():
```

```
return LaunchDescription([
    Node(package='my_cpp_pkg', executable='my_pub'),
    Node(package='my_cpp_pkg', executable='my_sub'),
```

])

launch를 실행시키면 generate_launch_description()을 호출하고, 반환된 리스트 안의 NNode들을 순서대로 명도의 프로세스로 실행한다. 그다음 모든 로그를 한 터미널에 모아서 출력한다.

A terminal window titled 'colcon build [1/1 done] [0 ongoing]' with a red header bar. The terminal shows a user 'yepunman' at a prompt '~\$' running 'cd ~/ros2_ws'. Then 'colcon build --packages-select my_cpp_pkg' is executed. The terminal shows 'source install/setup.bash' and 'ros2 launch my_cpp_pkg my_launch.py'. It then shows 'Starting >>> my_cpp_pkg' and 'Finished <<< my_cpp_pkg [0.93s]'. A summary line says 'Summary: 1 package finished [1.07s]'. Below that, log messages are shown: '[INFO] [launch]: All log files can be found below /home/yepunman/.ros/log/2026-09-15-13-24-13-118724-yepunman-10629', '[INFO] [launch]: Default logging verbosity is set to INFO', '[INFO] [my_pub-1]: process started with pid [10632]', '[INFO] [my_sub-2]: process started with pid [10633]', and several lines of '[my_sub-2] [INFO] [timestamp] [my_sub]: 받은 메시지: "Hello World!"'.

```
colcon build [1/1 done] [0 ongoing]
colcon build [1/1 done] [0 ongoing] 80x24
yepunman@yepunman:~$ cd ~/ros2_ws
colcon build --packages-select my_cpp_pkg
source install/setup.bash
ros2 launch my_cpp_pkg my_launch.py
Starting >>> my_cpp_pkg
Finished <<< my_cpp_pkg [0.93s]

Summary: 1 package finished [1.07s]
[INFO] [launch]: All log files can be found below /home/yepunman/.ros/log/2026-09-15-13-24-13-118724-yepunman-10629
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [my_pub-1]: process started with pid [10632]
[INFO] [my_sub-2]: process started with pid [10633]
[my_sub-2] [INFO] [1789446253.920422516] [my_sub]: 받은 메시지: "Hello World!"
[my_sub-2] [INFO] [1789446254.420373663] [my_sub]: 받은 메시지: "Hello World!"
[my_sub-2] [INFO] [1789446254.920470149] [my_sub]: 받은 메시지: "Hello World!"
[my_sub-2] [INFO] [1789446255.420340103] [my_sub]: 받은 메시지: "Hello World!"
[my_sub-2] [INFO] [1789446255.920461000] [my_sub]: 받은 메시지: "Hello World!"
[my_sub-2] [INFO] [1789446256.420427627] [my_sub]: 받은 메시지: "Hello World!"
```

launch로 실행한 함수는 과제 1-1의 코드 my_pub과 my_sub이다.

3. 커스텀 msg 정의 후 퍼블리시

정보를 전송할 때 문자와 숫자를 같이 보내야하는 경우가 있다. 일반적인 publisher를 사용하면 문자열로 숫자와 문자열을 다 같이 보내야하지만 커스텀 msg를 사용하면 미리 선언해준 형식으로 다른 자료형의 변수들을 한번에 전달 할 수 있다. 통신 메커니즘은 publisher/ subscriber와 동일하지만 커스텀 msg는 보내는 데이터를 구조화할 수 있게 해준다고 할 수 있다.

```
yepunman@yepunman:~/ros2_ws$ ros2 run my_cpp_pkg my_msg_pub
[INFO] [1789452122.527143866] [my_msg_pub]: 발행: id=0, label="hello world!"
[INFO] [1789452123.027137691] [my_msg_pub]: 발행: id=1, label="hello world!"
[INFO] [1789452123.527099081] [my_msg_pub]: 발행: id=2, label="hello world!"
[INFO] [1789452124.027137803] [my_msg_pub]: 발행: id=3, label="hello world!"
[INFO] [1789452124.527105119] [my_msg_pub]: 발행: id=4, label="hello world!"
[INFO] [1789452125.027120304] [my_msg_pub]: 발행: id=5, label="hello world!"
[INFO] [1789452125.527105878] [my_msg_pub]: 발행: id=6, label="hello world!"
[INFO] [1789452126.027119606] [my_msg_pub]: 발행: id=7, label="hello world!"
[INFO] [1789452126.527121153] [my_msg_pub]: 발행: id=8, label="hello world!"
[INFO] [1789452127.027111891] [my_msg_pub]: 발행: id=9, label="hello world!"
[INFO] [1789452127.527106102] [my_msg_pub]: 발행: id=10, label="hello world!"
[INFO] [1789452128.027100256] [my_msg_pub]: 발행: id=11, label="hello world!"
[INFO] [1789452128.527103578] [my_msg_pub]: 발행: id=12, label="hello world!"
[INFO] [1789452129.027131361] [my_msg_pub]: 발행: id=13, label="hello world!"
[INFO] [1789452129.527117001] [my_msg_pub]: 발행: id=14, label="hello world!"
[INFO] [1789452130.027119791] [my_msg_pub]: 발행: id=15, label="hello world!"
[INFO] [1789452130.527104560] [my_msg_pub]: 발행: id=16, label="hello world!"
^C[INFO] [1789452131.007086429] [rclcpp]: signal_handler(SIGINT/SIGTERM)
yepunman@yepunman:~/ros2_ws$
```

id, label을 받은 것을 각각의 데이터로 바로 사용할 수 있다.